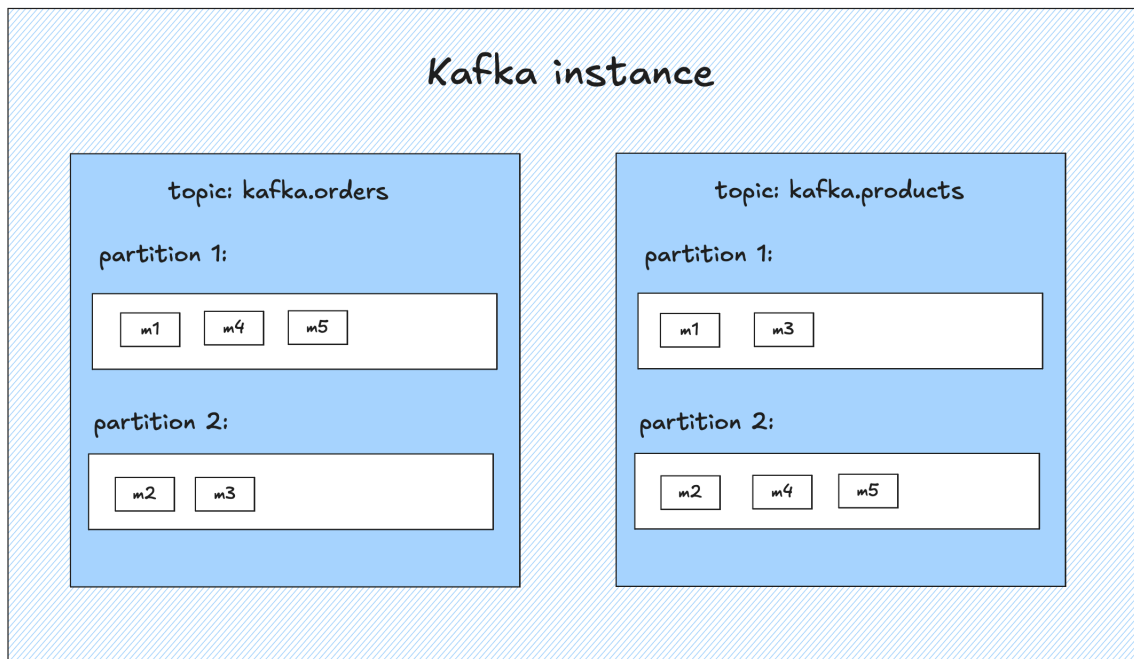




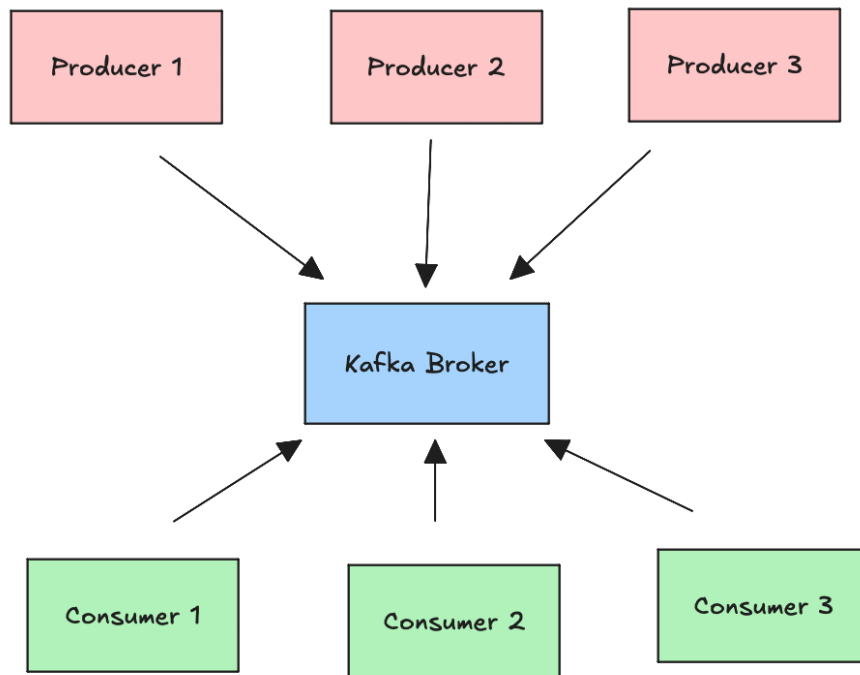
Apache Kafka

An open-source distributed event streaming platform with low-latency and high-throughput

1. **Message:** A piece of data collected, stored, distributed by Kafka
 - a. Can be a row, record, map or blob - treated as byte array
 - b. Can be an event in the event driven architecture
 - c. Can be a message in the message queues or in pub-sub architecture
 - d. size limit exists, default size is 1 MB (configurable)
 - e. Content
 - i. Key - used for partitioning
 - ii. Value - byte array hold content
 - iii. Time stamp - event time or ingestion time (configurable)
2. **Topic:** An entity which holds messages
 - a. Can be considered as files with papers or table with rows
 - b. Can be a Queue for similar messages (common structure/schema)
 - c. Example:- payment logs, video files, etc.,
 - d. A Kafka instance can have multiple topics
 - e. Support multiple producers and consumers concurrently
 - f. Each topic has multiple partitions with replications



3. **Broker:** A running Kafka instance listens on specific ports
- a. Receive messages from producers, stores, distributes to consumers
 - b. Keeps heartbeat with consumers to resend
 - c. Handles Subscription management
 - d. Manage topics, partitions and logs
 - e. Kafka cluster - multiple Kafka brokers with single Active Controller

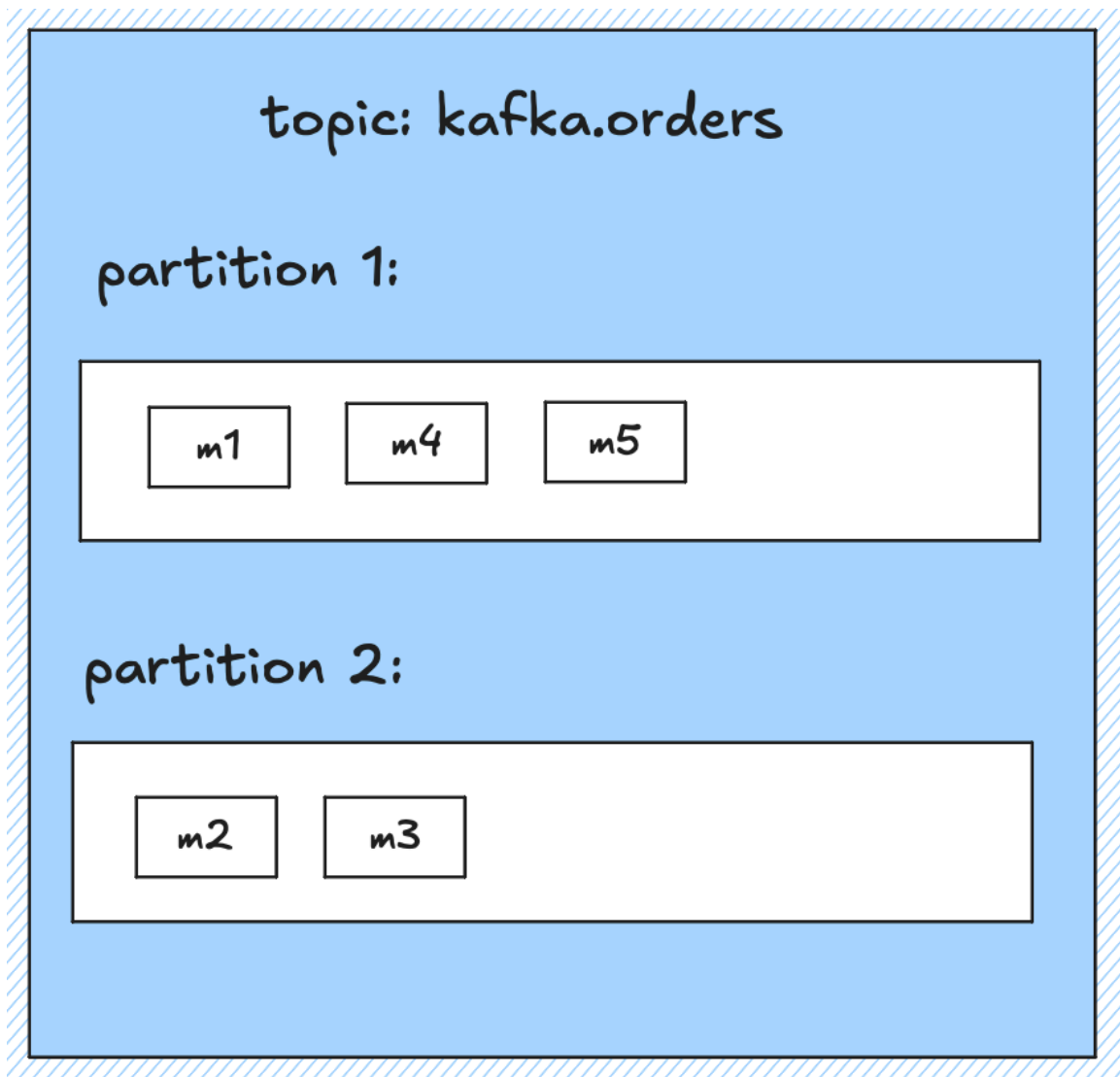


4. **Logs:** Physical files for storing data managed by brokers
 - a. Rolling files
 - b. Pruned periodically, default is 7 days
5. **Producer:** Any client that publishes data to Kafka broker
 - a. SDKs used to produce data
 - b. Multiple producers per topic is allowed
 - c. Identifies message key
 - d. Serialize the data to bytes
 - e. Can be Synchronous and Asynchronous
6. **Consumer:** Any client that consumes data from Kafka broker
 - a. Stream / Batch consume
 - b. SDKs are used to consume data

- c. Multiple consumers per topic
- d. Consumer groups for scaling workloads
- e. Deserialize bytes and recreate original data
- f. Manages the offset for data

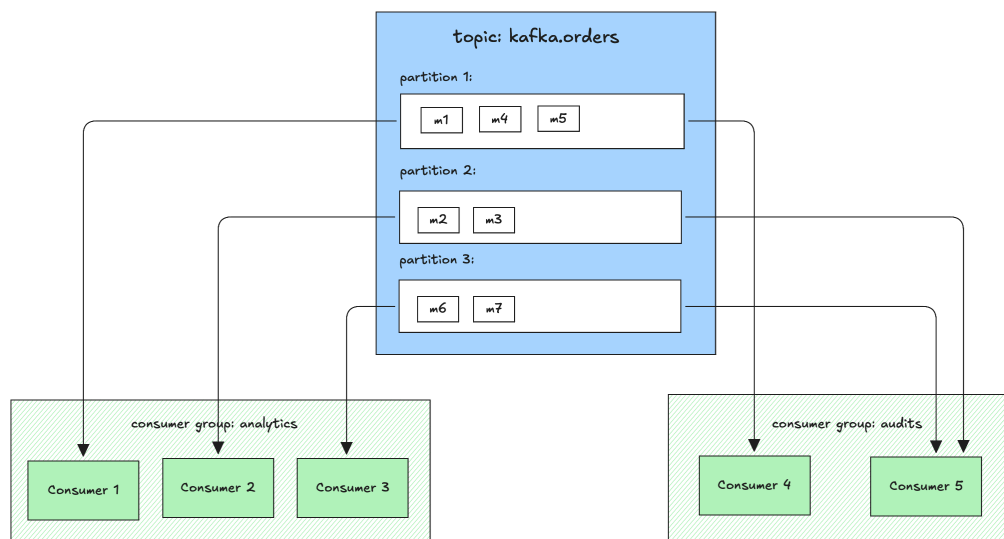
7. **Partition:** Partitions split a single topic logs across different nodes.

- a. A topic can have 1-n partitions
- b. Allows Kafka to scale by horizontal scalability
- c. A message is stored only in one partition
- d. Each partition has a leader broker - which manages them
- e. Can be replicated for fault tolerance
- f. Same key = same partition using hash function



8. **Consumer Groups:** A group of consumers who share a topic workload

- a. Each message will be sent to only one consumer in a group
- b. A single message can be consumed by multiple consumer groups if they subscribed to that topic.
- c. Consumer split workload through partitions
- d. Number of partitions $\geq$ Number of consumers (if there are more consumers there may be idle consumers)
- e. Brokers track and rebalance consumers in a group with the help of heart beats



9. **Offset:** Number to track message consumption by consumer
- a. Broker keeps track of what is sent and acknowledged
 - b. Broker resends messages in case of failure or timeout for ack
 - c. Ensures at least one delivery to a consumer group